

# ENEE 633 - Statistical Pattern Recognition Project 02

Kiyan Amirian

May 15, 2025

In this report, we present the results of classifying the MNIST dataset using Support Vector Machines (SVM), Logistic Regression, and Convolutional Neural Networks (CNNs) in Part 1, as well as the classification of the 10 Monkey Species dataset using transfer learning. The dataset was obtained through the `keras.datasets` module. Figure 1 provides a visual overview of a few sample images from the MNIST dataset.

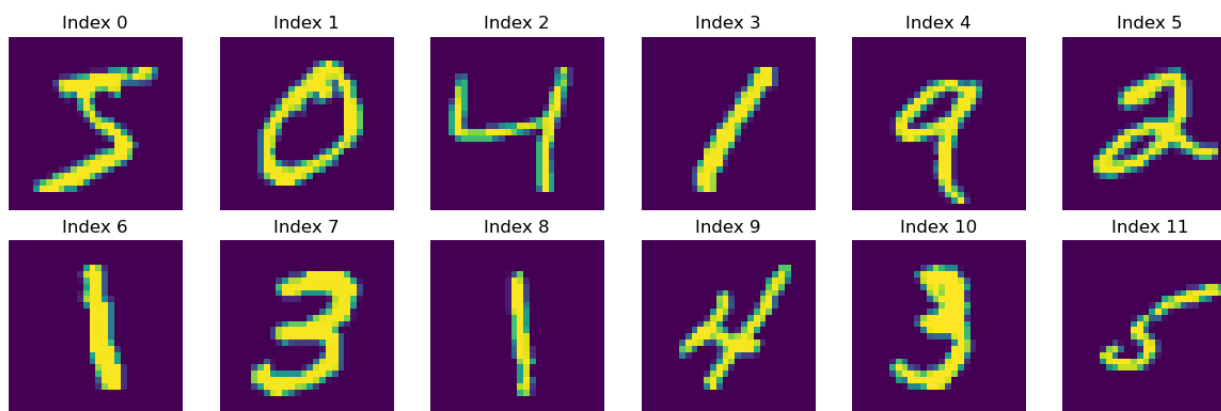


Figure 1: pictures of handwritten digits.

To examine how evenly the digits 0 through 9 are represented across the training and test sets, I created class distribution histograms using Seaborn's `countplot` function—these are shown in Figures 2 and 3. Additionally, to improve computational efficiency, I limited the training set to 10,000 samples rather than using the full set of 60,000 images.

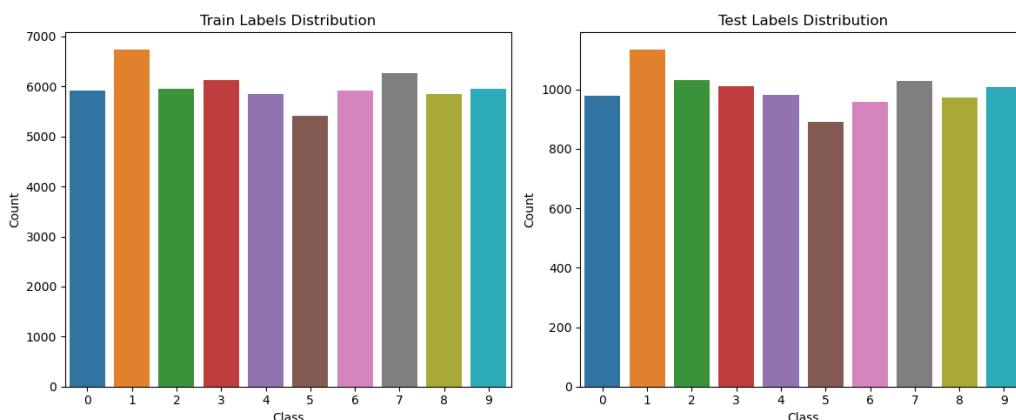


Figure 2: histogram of labels (make it 2-1)

## 1.1 SVM

To implement Support Vector Machines (SVM) on the dataset, I used the LIBSVM library. The models were trained using `svm-train()` and evaluated using `svm-predict()`. Three kernel types were tested: linear, polynomial, and radial basis function (RBF). As shown in Table 1, the RBF kernel achieved the highest accuracy at 96.19%, followed by the polynomial kernel at 95.21%. The linear kernel performed reasonably well with an accuracy of 91.39%, though it was outperformed by the nonlinear kernels. Training each model on the full MNIST dataset took over five minutes.

| Kernel Type | Accuracy (%) |
|-------------|--------------|
| Linear      | 91.39        |
| Polynomial  | 95.21        |
| RBF         | 96.19        |

To test PCA and LDA, I found that both methods were taking too long to run regardless of my approach. Eventually, I was able to perform PCA and LDA by normalizing the data first. Interestingly, normalization also improved the performance of the baseline SVM. Below are the results after applying PCA and LDA:

| Kernel     | Raw SVM Accuracy (%) | PCA SVM Accuracy (%) | LDA SVM Accuracy (%) |
|------------|----------------------|----------------------|----------------------|
| Linear     | 92.93                | 93.24                | 89.33                |
| Polynomial | 96.11                | 96.62                | 90.96                |
| RBF        | 96.61                | 97.1                 | 92.15                |

Normalization alone yields strong baseline results, especially for the RBF kernel (96.19%). PCA (50 components) offers marginal improvements across all kernels, with polynomial and RBF reaching 96.62% and 97.10%, indicating that PCA effectively reduces noise while preserving key features. LDA (9 components), though supervised, performs slightly worse—likely due to over-compression of the data, limiting its ability to capture complex decision boundaries. Overall, PCA provides modest but consistent accuracy boost over raw normalized input.

## 1.2 Logistic regression

I implemented logistic regression in two ways: using scikit-learn and from scratch with NumPy. The scikit-learn model achieved 91.16% accuracy on raw MNIST images, 89.96% with PCA, and 86.15% with LDA. In the custom implementation, the accuracy was slightly lower: 87.09% without dimensionality reduction, 88.72% with PCA, and 86.76% with LDA. The lower performance in the from-scratch version may be due to simplifications in the training loop or

optimization. In both cases, PCA slightly improved accuracy, while LDA tended to reduce it. Logistic regression was implemented using `sklearn.linear_model.LogisticRegression` with L1 penalty, the "saga" solver, and a tolerance of 0.1. As shown in Table 1, PCA had little impact on accuracy, while LDA caused a noticeable drop, likely due to loss of discriminative features.

Table 1: Logistic Regression (scikit-learn implementation)

| Method         | Accuracy (%) | Method         |
|----------------|--------------|----------------|
| Raw Input      | 91.16        | Raw Input      |
| PCA (50 comps) | 89.96        | PCA (50 comps) |
| LDA (9 comps)  | 86.15        | LDA (9 comps)  |

The custom model used softmax, cross-entropy loss, and gradient descent. As shown in Table 2, results were slightly lower than the scikit-learn version. PCA improved performance slightly, but LDA again reduced accuracy, consistent with the packaged model's behavior.

Table 2: Logistic Regression (from-scratch implementation)

| Method         | Accuracy (%) | Method         |
|----------------|--------------|----------------|
| Raw Input      | 87.09        | Raw Input      |
| PCA (50 comps) | 88.72        | PCA (50 comps) |
| LDA (9 comps)  | 86.76        | LDA (9 comps)  |

This is also the result of the cross-entropy loss vs epoch for logistic regression from scratch:

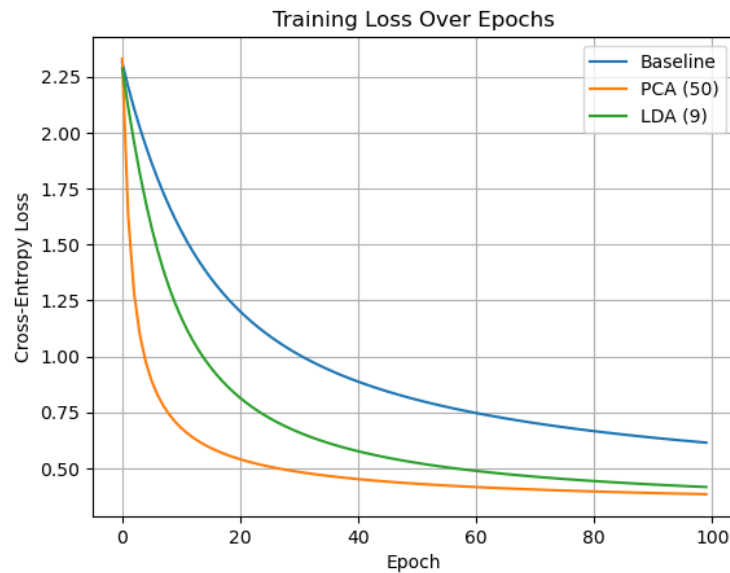


Figure 3: cross entropy loss vs epoch for logistic regression from scratch

## 1.3 Deep learning

I have used Keras for deep learning and below is my neural network structure with total of 79,306 trainable parameters:

| Layer (type)                   | Output Shape       | Param # |
|--------------------------------|--------------------|---------|
| conv2d_6 (Conv2D)              | (None, 26, 26, 64) | 640     |
| activation_8 (Activation)      | (None, 26, 26, 64) | 0       |
| max_pooling2d_6 (MaxPooling2D) | (None, 13, 13, 64) | 0       |
| conv2d_7 (Conv2D)              | (None, 11, 11, 64) | 36,928  |
| activation_9 (Activation)      | (None, 11, 11, 64) | 0       |
| max_pooling2d_7 (MaxPooling2D) | (None, 5, 5, 64)   | 0       |
| conv2d_8 (Conv2D)              | (None, 3, 3, 64)   | 36,928  |
| activation_10 (Activation)     | (None, 3, 3, 64)   | 0       |
| max_pooling2d_8 (MaxPooling2D) | (None, 1, 1, 64)   | 0       |
| flatten_2 (Flatten)            | (None, 64)         | 0       |
| dense_4 (Dense)                | (None, 64)         | 4,160   |
| dense_5 (Dense)                | (None, 10)         | 650     |
| activation_11 (Activation)     | (None, 10)         | 0       |

Figure 4: structure of the CNN.

Below can be seen the image of accuracy vs epoch and loss vs epoch for both training set and validation set, the accuracy has reached to 95.48 percent which is the highest thus far.

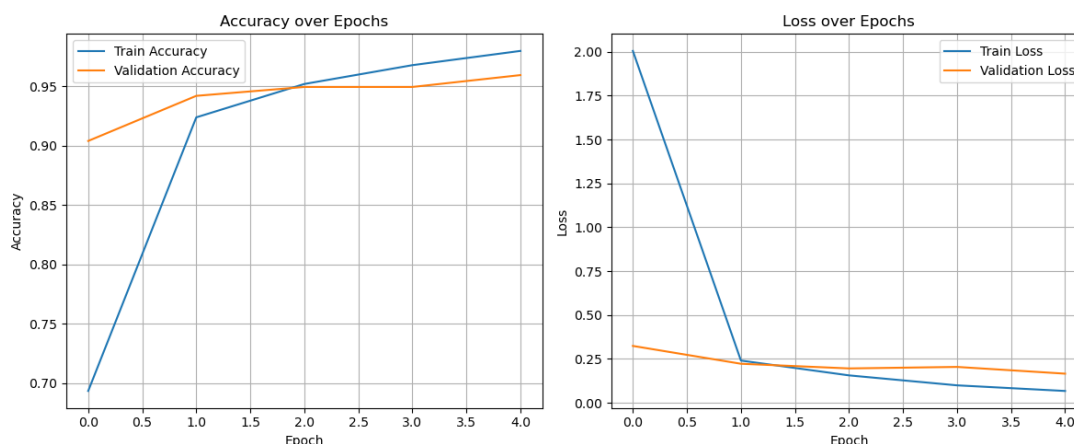


Figure 5: Training and validation accuracy (left) and loss (right) over 5 epochs.

## 2.1 Custom CNN Classifier

I first built a custom convolutional neural network (CNN) using Keras's Sequential API to classify monkey species into 10 classes. The architecture included three convolutional layers with ReLU activation and max pooling, followed by two dense layers with 120 and 84 units, and a final softmax output layer. I trained the model on 224×224 images for 5 epochs using the Adam optimizer with a learning rate of 1e−4 and categorical cross-entropy loss. I also applied data augmentation using ImageDataGenerator, which included preprocessing and horizontal flipping.

The architecture of the custom CNN can be seen below:

| Layer (type)                   | Output Shape         | Param #    |
|--------------------------------|----------------------|------------|
| conv2d_2 (Conv2D)              | (None, 222, 222, 32) | 896        |
| max_pooling2d_2 (MaxPooling2D) | (None, 111, 111, 32) | 0          |
| conv2d_3 (Conv2D)              | (None, 109, 109, 64) | 18,496     |
| max_pooling2d_3 (MaxPooling2D) | (None, 54, 54, 64)   | 0          |
| flatten_1 (Flatten)            | (None, 186624)       | 0          |
| dense_5 (Dense)                | (None, 120)          | 22,395,000 |
| dense_6 (Dense)                | (None, 84)           | 10,164     |
| dense_7 (Dense)                | (None, 10)           | 850        |

Figure 6: structure of the CNN.

And below can be seen the training and validation accuracy over 5 epochs.

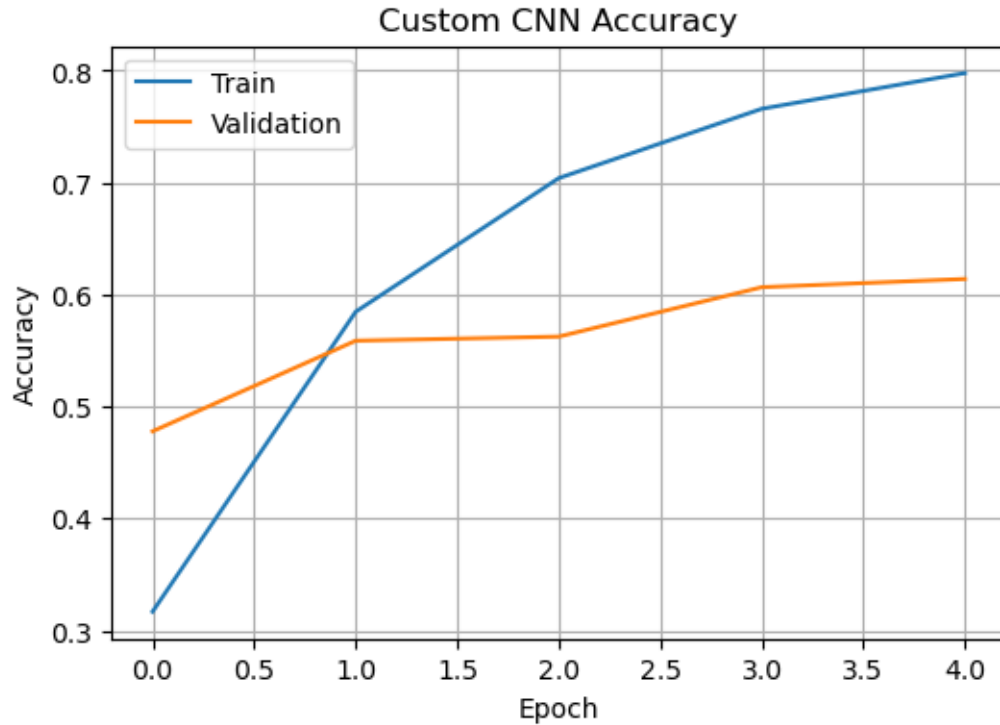


Figure 7: Training and validation accuracy. Note that epoch 0 is 1.

The accuracy reached only 63.24%, as shown in the plot.

## 2.2 Transfer Learning with Frozen VGG19

To improve performance, I applied transfer learning using a pretrained VGG19 model from `keras.applications`. I froze the convolutional base and added a new classification head consisting of a dense layer with 1024 units and ReLU activation, a dropout layer with a rate of 0.4, and a softmax output layer. I compiled the model using stochastic gradient descent (SGD) with a learning rate of  $1e-4$  and trained it for 5 epochs. This transfer learning approach significantly improved validation accuracy compared to the custom CNN.

below can be seen the training and validation accuracy over 5 epochs.

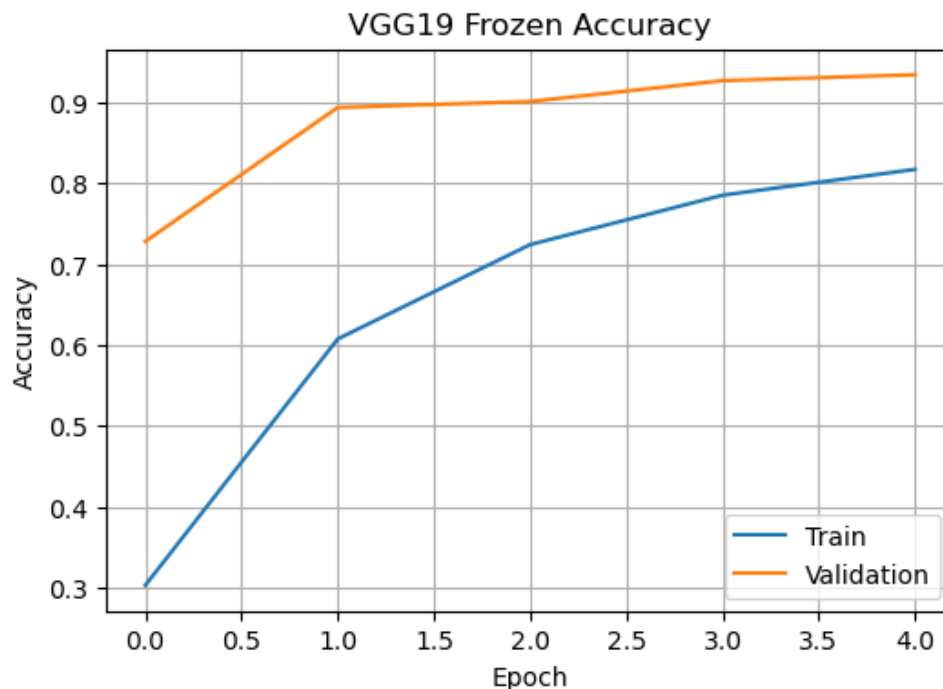


Figure 8: Training and validation accuracy. Note that epoch 0 is 1.

The validation accuracy here has gone much higher to 93.38 percent. Notably, validation accuracy exceeded training accuracy, which I discussed later.

## 2.3 Fine-Tuning VGG19

In the final experiment, I fine-tuned the entire VGG19 model by unfreezing all layers and training it end-to-end. I reduced the learning rate to  $1e-5$  to prevent large updates and overfitting. I trained the model for another 5 epochs using the same setup, which resulted in the best classification accuracy among all the approaches I tested.

below can be seen the training and validation accuracy over 5 epochs.

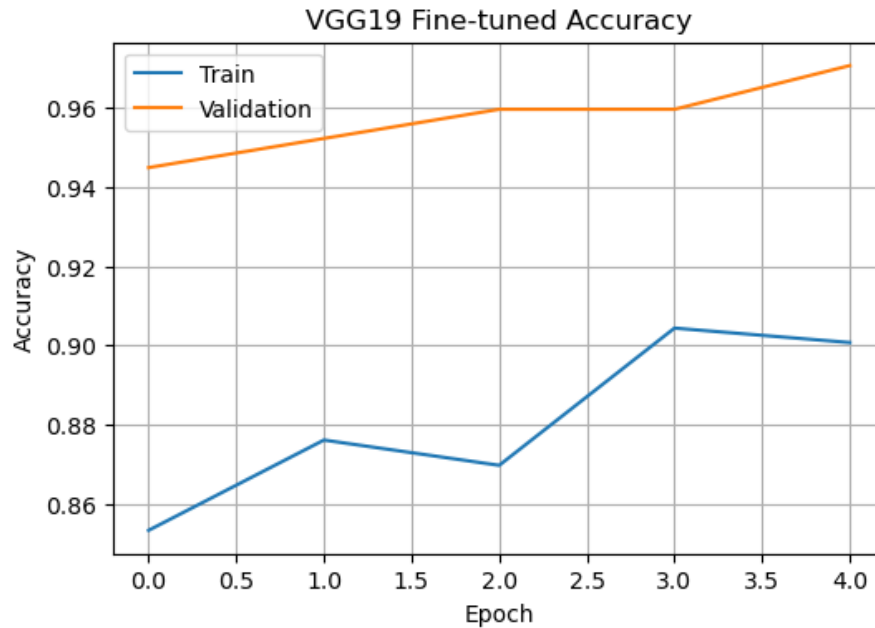


Figure 9: Training and validation accuracy. Note that epoch 0 is 1.

As can be seen The accuracy improved to 97.06%.

## Discussion

In this part, I tested three models to classify monkey species: a custom CNN, transfer learning with frozen VGG19, and fine-tuned VGG19. The custom CNN achieved a validation accuracy of 63.24% after 5 epochs. While data augmentation helped, the model's limited depth and size likely affected its performance. Using VGG19 with frozen layers significantly improved accuracy, reaching 93.38% validation accuracy. This suggests that pretrained ImageNet features are highly effective. Interestingly, validation accuracy was higher than training accuracy, likely because of data augmentation during training, which makes the model see harder versions of the images. Finally, fine-tuning all layers of VGG19 improved the result even more, reaching 97.06%. Unfreezing the layers allowed the model to better adapt to the monkey dataset. Overall, moving from a simple model to a pretrained and fine-tuned one clearly improved the accuracy.